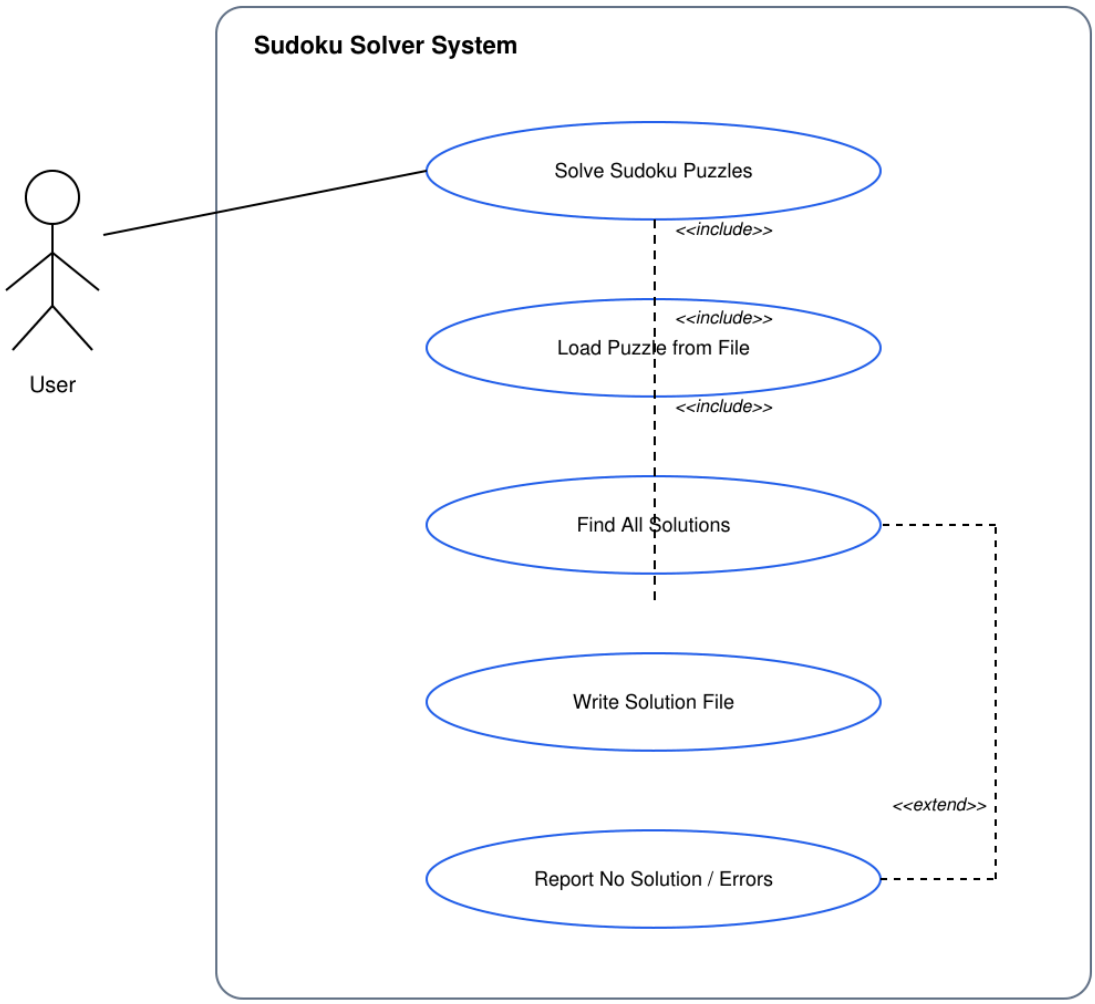
Sudoku Solver — Assignment 5 Use Case

Modified from Jacob Dean Lehrman's Assignment 4 model | Peter Lauman | 29 Oct 2026



Input: puzzle1.txt through puzzle5.txt (9x9 grids, underscore for blanks)

Output: solution1.txt through solution5.txt; all grids and final solution count

Alternative: a single "No solution found." line when no valid solution exists

Diagram revised to reflect Assignment 5 file output and validation requirements.

Use Case Specifications

Use Case 1 — Solve Sudoku Puzzles

Brief description: Primary actor: User. Brief description: Run the program to process five Sudoku puzzle files and generate matching solution files.

Step-by-step event flow: 1. User starts the program. 2. System processes puzzle1.txt through puzzle5.txt. 3. For each file, system loads and validates the puzzle. 4. System finds all valid completions by recursive backtracking. 5. System writes the results and count or a no-solution message. 6. System continues until all files are processed. Detailed description: Precondition: program has permission to access its working directory. Success: each readable valid puzzle receives a solutionN.txt containing every completed 9x9 grid followed by Solutions found: n, or a single No solution found. line. Failed input/output access is reported; the program continues to subsequent puzzles.

Use Case 2 — Load Puzzle from File

Brief description: Included use case: read and validate a puzzle grid.

Step-by-step event flow: 1. Open named file. 2. Read exactly 81 values. 3. Accept only digits 1–9 and underscores. 4. Reject missing, unexpected, or extra values. 5. Check starting-clue consistency. Detailed description: An invalid or contradictory input is not sent through ordinary search. Invalid file contents produce an error; contradictory clues yield no solution.

Use Case 3 — Find All Solutions

Brief description: Included use case: recursively enumerate valid Sudoku completions.

Step-by-step event flow: 1. Skip filled cells. 2. Try digits 1–9 for each empty cell. 3. Check row, column, and 3x3 box. 4. Recurse after placing a valid digit. 5. On a full grid, record the solution. 6. Backtrack to discover alternatives. Detailed description: The search does not stop after its first solution. It restores each tried cell after recursion, so all possible completions can be considered.

Use Case 4 — Write Solution File

Brief description: Included use case: save the results for each puzzle. Step-by-step event flow: 1. Open solutionN.txt. 2. For each complete grid, write nine space-separated rows. 3. Separate successive solutions. 4. Append Solutions found: n. 5. Close the file. Detailed description: Output files use the same cell format as input puzzles, without console-only decorative separators. The final count line appears after all solutions.

Use Case 5 — Report No Solution / Errors

Brief description: Conditional use case: communicate unsuccessful searches or file errors.

Step-by-step event flow: 1. If the puzzle has zero completions, write No solution found. as the only output line. 2. If an input file cannot be loaded or an output file cannot be created, report the error. 3. Continue processing other puzzles. Detailed description: The no-solution case is distinct from a missing or invalid file. For valid but inconsistent clues, the output file contains only the required no solution line.